

Contents

Admin API Guide	1
Auth: no separate admin sign-up	1
Promoting the 5 of you to ADMIN	2
Admin-only endpoints	2
Clubs	2
Gameweeks	2
Fixtures	3
Fixture Results	3
Players	3
Player Prices	4
Scoring	4
Notifications	5
Fantasy Teams	5
Notes for the frontend build	5

Admin API Guide

Reference for building the admin website: what “admin” means on this backend, how admins log in, and every endpoint that requires the ADMIN role.

Base URL: <https://gpllivebackend.onrender.com> (same deployed backend the mobile app uses - no separate admin backend).

Auth: no separate admin sign-up

There is **no admin registration form**, and the admin website shouldn’t build one. Here’s why and how it actually works:

- Every admin already has a normal account in the users table (they signed up in the mobile app with their Gmail address, same as any other user).
- The users table has a role column (USER or ADMIN, defaults to USER). Whether someone can call an admin endpoint is entirely determined by this column - there’s no separate admin identity or password.
- The admin website should have a **login form only**, calling the exact same endpoint the mobile app uses:

POST /auth/login

```
{ "email": "admin@gmail.com", "password": "their-existing-password" }
```

Response:

```
{ "token": "eyJ...", "username": "...", "role": "ADMIN" }
```

Store the JWT and send it as Authorization: Bearer <token> on every request after this, same as the mobile app.

- The role field in the response is what the frontend should check to decide whether to show the admin dashboard or a “not an admin” screen. This is a UX convenience only - the backend independently re-checks the role on every admin-only endpoint (@PreAuthorize("hasRole('ADMIN')")), so there’s no security risk in trusting role for

UI decisions; a non-admin can never actually get an admin response back even if they bypass the UI check. (GET /auth/users/me also returns role if you need to re-check it later without a fresh login.)

Promoting the 5 of you to ADMIN

Since there's no self-serve way to become an admin (intentionally - a sign-up-your-own-admin-account flow would be a security hole), the first batch has to be done directly in the database. In Neon's SQL console, for each teammate's existing email:

```
UPDATE users SET role = 'ADMIN' WHERE email = 'teammate1@gmail.com';
UPDATE users SET role = 'ADMIN' WHERE email = 'teammate2@gmail.com';
-- etc for all 5
```

No redeploy or re-login needed for it to take effect - the backend re-reads role from the database on every request (it's not baked into the JWT), so the change applies on that person's very next API call.

If down the line you want to add/remove admins without touching the database directly, that would need a new admin-only endpoint (e.g. PATCH /users/{id}/role) - not built yet, ask if you want it added.

Admin-only endpoints

All of these require Authorization: Bearer <token> from a logged-in ADMIN. Calling them as a non-admin (or logged out) returns 403 Forbidden.

Clubs

Method	Path	Body
POST	/clubs	ClubRequest (below)
PUT	/clubs/{id}	ClubRequest

```
// ClubRequest
{
  fullName: string;      // required
  shortName: string;     // required
  logoUrl: string;       // required
  homeGround: string;    // required
  foundedYear: number;   // required
  city: string;          // required
  clubStatus: "ACTIVE" | "RELEGATED" | "WITHDRAWN" | "SUSPENDED"; // required
}
```

Gameweeks

Method	Path	Body
POST	/gameweeks	GameweekRequest
PUT	/gameweeks/{id}/set-current	none

```
// GameweekRequest
{
  season: string;           // required, e.g. "2025/2026"
  gameweekNumber: number;   // required
  startDate: string;        // required, ISO datetime
  endDate: string;          // required, ISO datetime
  deadline: string;         // required, ISO datetime - squad lock time
}
```

Fixtures

Method	Path	Body
POST	/fixtures	FixtureRequest
PATCH	/fixtures/{id}/status	raw status string (below)

```
// FixtureRequest
{
  homeClubId: number;       // required
  awayClubId: number;       // required
  gameweekId: number;       // required
  matchDate: string;        // required, ISO datetime
  venue: string;            // required
}

// PATCH /fixtures/{id}/status body - just the raw enum value as JSON, e.g.:
"LIVE"
// one of: "SCHEDULED" | "LIVE" | "FINISHED" | "POSTPONED"
```

Fixture Results

Method	Path	Body
POST	/fixture-results	FixtureResultsRequest

```
// FixtureResultsRequest
{
  fixtureId: number;
  homeScore: number;
  awayScore: number;
  homePossession?: number; // optional, percentage
  awayPossession?: number; // optional, percentage
}
```

Players

Method	Path	Body
POST	/players	PlayerRequest

Method	Path	Body
PUT	/players/{id}	PlayerRequest

```
// PlayerRequest
{
  fullName: string;           // required
  clubId: number;             // required
  jerseyNumber: number;       // required
  position: "GK" | "DEF" | "MID" | "FWD"; // required
  photoUrl?: string;          // optional
  nationality: string;         // required
  status: "AVAILABLE" | "SUSPENDED" | "INJURED" | "INACTIVE" | "INTERNATIONAL_DUTY"; // re
}
```

Player Prices

Method	Path	Body
POST	/player-price	PlayerPriceRequest

```
// PlayerPriceRequest
{
  playerId: number;           // required
  gameweekId: number;         // required
  price: number;              // required, decimal
}
```

Scoring

Method	Path	Body
POST	/scoring/stats	PlayerGameWeekStatsRequest
POST	/scoring/calculate-all/{gameweekId}	none - triggers scoring calculation for every fantasy team that gameweek

```
// PlayerGameWeekStatsRequest
{
  playerId: number;           // required
  fixtureId: number;          // required
  minutesPlayed: number;       // required
  goalsScored: number;         // required
  assists: number;             // required
  cleanSheet: boolean;         // required
  yellowCard: number;          // required, count (0, 1, 2)
  redCard: boolean;            // required
  saves: number;               // required
}
```

Notifications

Method	Path	Body
POST	/notifications	NotificationRequest - sends a notification to one specific user

```
// NotificationRequest
{
  userId: number;    // required - the recipient
  message: string;   // required
  type: "DEADLINE" | "RANK" | "GOAL" | "CAPTAIN"; // required
}
```

Fantasy Teams

Method	Path	Body
GET	/fantasy-teams	none - lists every user's fantasy team (admin overview)

Notes for the frontend build

- All the POST/PUT endpoints above return the created/updated resource as JSON (e.g. POST /clubs returns the new ClubResponse), so you can refresh the UI straight from the response instead of re-fetching.
- Validation errors come back as 400 with a message field describing which field failed (e.g. "fullName: must not be blank").
- A duplicate/conflicting write (e.g. a club that already exists) comes back as 409 with a message explaining the conflict.
- There's currently no dedicated "list all users" or "list all players admin view" endpoint beyond what's listed above - the public GET endpoints on each resource (/clubs, /players, /fixtures, etc.) work fine for admins too, they're just not admin-restricted since regular users need them too.